

How HACKPROOF Calculates Terminal Output

Line-by-line guide explaining the exact Git commands, mathematical formulas, and forensic logic behind every line in the terminal audit report.

1. The Header Banner Verdict

```
HACKPROOF AUDIT REPORT: ✔ ALL CHECKS PASSED (VERIFIED GENUINE WORK)  
Target Repository : anantapathak8/Ureckon-27
```

How it calculates the verdict:

- The engine executes all 23 checks across Claim, System, and Server planes.
- Each check returns a severity (hard_flag, flag, or info) and a boolean passed.
- **Decision Matrix:**
 - If failed == 0: ✔ ALL CHECKS PASSED (Genuine live build).
 - If any hard_flag failed (e.g. backdated parent commits, outsider key, concealed code): ❌ CRITICAL INTEGRITY THREAT DETECTED.
 - If only flag failed (e.g. timezone mismatch, commits before kickoff): ⚠ SUSPICIOUS ACTIVITY DETECTED.

2. Section [1]: Commit Signature Verification (GPG / SSH)

```
[1] COMMIT SIGNATURE VERIFICATION (GPG / SSH)  
• Coverage: 2/2 commits verified (100%)  
• ⚠ UNSIGNED COMMITS (Missing Cryptographic Proof):  
  - Commit 447827b authored by "Ananta Pathak <ananta@...>"  
    → Missing GPG signature! (Pushed from an unregistered laptop)
```

How it extracts and computes signatures:

- **Git Command:** `git log --format="%H%x00%an%x00%ae%x00%G?%x00%GK%x00%GS"`
- **Porcelain Flags Read:**
 - %G?: Git signature status (G = Good valid key, B = Bad/broken, U = Untrusted key, N = No signature).
 - %GK: 16-character or 40-character Key ID used to sign.
 - %ae: Git author email.
- **The Math for Coverage:** $\text{Coverage Ratio} = \frac{\text{Verified Signed Commits}}{\text{Total Commits}} \times 100\%$ A commit is only counted as Verified if %G? == 'G' **AND** the signing key %GK is registered to that member in hackproof.db.
- **Unsigned Commits:** Collected from every commit where %G? == 'N'.
- **Unregistered Keys:** Flagged when %G? == 'G', but %GK belongs to someone outside the team roster.
- **Identity Mismatch:** Flagged when key %GK belongs to Alice, but author email %ae is Bob.

3. Section [2]: File Concealment & .gitignore Blame Table

[2] FILE CONCEALMENT & PRE-BUILT CODE INSPECTOR (§8.f)

• Status: Clean (No hidden files, no pre-built code staging, no index manipulation).

• FORENSIC .GITIGNORE BLAME VIEW (§8.f):

	Line	Rule / Pattern	Author	Committed Date & Time	Timing	
Status						
CLEAN	1	node_modules/	Ananta Pathak	2026-08-24 00:18:16 +05	[LATE +2.8h]	✓
	4	.env.local	Ananta Pathak	2026-08-24 00:18:16 +05	[LATE +2.8h]	⚠
SUSP						

How Status: Clean is calculated:

• Disk vs Git Ratio: os.walk counts all source files on disk; compares with git ls-files. Must be ≥ 90% tracked.

• Index Flags: Runs git ls-files -v to detect stealth --skip-worktree (marker S) or --assume-unchanged (marker M).

• Local Excludes: Inspects .git/info/exclude and core.excludesFile overrides that never get pushed to GitHub.

TABLE COLUMN	GIT COMMAND / TELEMETRY SOURCE	CALCULATION & FORENSIC FORMULA
Line	git blame --line-porcelain -- .gitignore	Line number index of the rule in the file.
Rule / Pattern	Tab content line in porcelain output (...)	The exact regex/glob rule string (e.g. node_modules/, .env.local).
Author	author header in blame block	Git committer name who introduced that exact line.
Committed Date	author-time (epoch) & author-tz	Converted from UNIX epoch: datetime.fromtimestamp(author_time, tz).
Timing	git rev-list --max-parents=0 HEAD	Finds initial commit epoch (\$T_0\$). Elapsed = (author_time - T0) / 3600 hours If in initial commit: [INIT]. If later commit: [LATE +X.Xh].
Status	Pattern taxonomy + git check-ignore -v	• ✓ CLEAN: Benign build artifacts (node_modules, dist, __pycache__). • ⚠ SUSP: Sensitive config/source patterns (.env.local, *.py, _build/). • 🚫 CONCEALED: The rule hides uncommitted code files physically on disk!

4. Section [3]: Build Window & Timestamps (§8.d / §8.i)

[3] HACKATHON BUILD WINDOW & TIMESTAMPS (§8.d / §8.i)

• First Commit : 2026-08-23T21:28:10+05:30 (Commit 23c8cc1)

• Latest Commit : 2026-08-24T00:18:16+05:30 (Commit 447827b)

• Active Span : 2h 50m across 4 commits

• Status: Clean (Monotonic chronology, consistent timezones, natural human intervals).

How timestamp forensics are calculated:

- **First & Latest Commit:** Parsed via `git log --reverse --format="%H%x00%ci"`. First is index 0; Latest is index -1.
- **Active Span:** `Delta = (Latest_Epoch - First_Epoch) → Hours & Minutes`
- **Parent Monotonicity Check:** For every commit C and parent P :
`IsValid = CommitterDate(C) ≥ CommitterDate(P)` If $C < P$, commit was backdated or rebased! Flags: 
`claim.parent_monotonicity`.
- **Timezone Consistency:** Counts occurrences of every timezone offset (e.g. `+05:30`: 4 commits). Dominant offset becomes baseline. Any commit with differing offset is flagged:  `claim.timezone_consistency` (catches offshore ghost developers).
- **Typing Speedrun / Burst Check:** Measures interval ΔT between consecutive commits. If $\Delta T < 2.0 \text{ ext}\{\text{seconds}\}$ while adding >100 lines of code, it flags automated code copy-pasting.

5. Judge Takeaway Verdict

JUDGE TAKEAWAY:  **Verified genuine build provenance. Clear for hackathon judging.**

How the final recommendation is selected:

- if `is_clean`:  Verified genuine build provenance. Clear for hackathon judging.
- elif `hard_flags`:  High-risk cheating indicators detected. Prompt team for explanation.
- else:  Incomplete signature coverage. Verify commits with the team.